Planck-99

TinyML Binary Malware Classifier for Embedded Linux & IoT — Technical Brief

Ziad Salah • Division-36 • v1

98.88%	
Accuracy — internal set (§03)	**96.28%**
Accuracy — unseen IoT (OOD)	**34 ns**
Median C-kernel latency	**27 KB**
Model binary footprint	**0.03 MB**
Working set per classification	

Executive summary. Planck-99 is a deterministic, integer-only binary malware classifier for embedded Linux and IoT. The model was trained **exactly once** (on commodity CPU, ≈13 minutes) and evaluated **three times without retraining**: in-sample calibration on the 5,550-trace training corpus (98.88%), and two fully external, unseen corpora — OOD IoT syscalls (96.28%) and Linux static-ELF malware (92.01% detection rate). Inference is a fixed 32-dimension integer decision over a 5-tree ensemble: median latency 34–62 ns, per-classification working set ≈0.03 MB, zero FPU, zero network, zero runtime — fit for a single SRAM-less CPU and verifiable bit-for-bit from published traces. Every verdict carries a JSON evidence trail (path node IDs and threshold references/hashes — never model internals).

01 What Planck-99 Is

Planck-99 is a deterministic, integer-only binary classifier that ingests a host-level syscall trace and returns a single verdict: Malware or Benign. It compiles to a single dependency-free C translation unit — a 462-line inference kernel plus a generated 625-line model header — with no ML runtime, no interpreter, and no external dependencies. It runs entirely in userspace, with zero floating-point operations on the inference path. Every verdict is accompanied by a JSON audit trail recording what the engine did to reach it.

Determinism is engineered, not assumed.

For any pinned engine version, identical input always yields an identical verdict — on every run, on every build, on every architecture. That guarantee rests on four specific design facts:

- **Fixed-width integer math only.** The inference path performs integer comparisons only — no floating-point rounding, no math library, no FMA — so outputs are bit-exact rather than approximately equal.
- **A fixed input.** The classifier consumes exactly one 32-dimensional feature vector; nothing else is sampled, randomized, or threaded through the decision.
- **Frozen-at-build-time logic.** The trained model is compiled into the binary as static threshold comparisons, not interpreted or updated at runtime.
- **No hidden state.** The decision path uses no mutable globals, no heap allocation, no threads, and no dependence on clocks or timing.

The scope of this claim is precise: determinism guarantees that a verdict is *reproducible, replayable, and attributable*. It is not a claim that detection is complete, nor that crafted inputs cannot evade the model.

Verifiable without exposing the source.

Because inference is a pure function, determinism can be checked with no code disclosure: replay a published trace and confirm the verdict and audit trail match bit-for-bit, or reproduce the binary from the published model header and compare hashes. The JSON audit trail is a per-decision *evidence artifact*, not the decision function. It records what the engine did — the input feature vector (or its

hash), the **decision-path node identifiers and the threshold references/hashes** consulted for that verdict, the verdict, and the pinned engine version — never how the model was built or trained. Model logic is reserved for a restricted deep-audit tier; public assurance rests on reproducible black-box results.

The evidence tiering is explicit: (i) **public layer** — traces, verdicts, audit trails, batch statistics, and the model header, all hash-pinned; (ii) **restricted deep-audit layer** — weights/threshold values and the trainer, disclosed under NDA or a regulatory deep-audit process (e.g., CRA Annex IV essential-requirements audits, NIS2 supervision). This split keeps day-to-day assurance public while preserving the option of full independent review.

The design target is first-line detection on resource-constrained embedded Linux and IoT devices — hardware classes that rule out traditional AV, cloud pipelines, or GPU-accelerated inference. Planck-99 v1 is the initial release; the limitations section (07) documents what future versions will improve.

02 Architecture Decisions & Rationale

Why a decision-tree ensemble, not a neural network?

The deployment constraint is a C compiler and integer arithmetic — nothing else. Neural networks require matrix multiplication in floating-point or dedicated quantization runtimes (TFLite, ONNX RT) that add hundreds of KB of dependency before a single weight is loaded. A pruned decision-tree ensemble compiles to a nested if-else structure — roughly four KB of C source for the ensemble logic itself, and 27 KB for the complete deployed binary (462-line inference kernel plus 625-line generated model header; see section 04). Every path through the trees is fixed and known before deployment, so the full decision structure is auditable by the vendor statically and replayable by anyone from published traces.

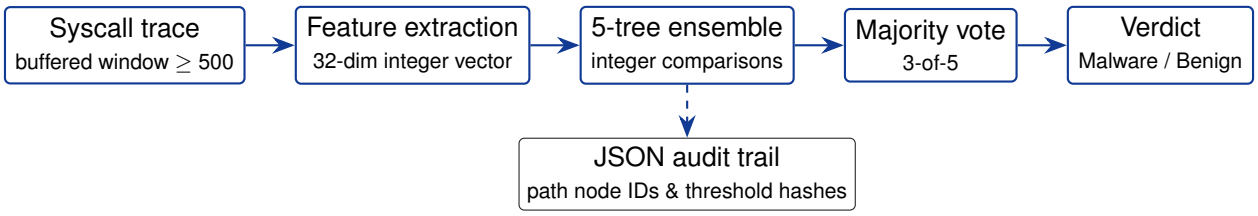

Figure 1: Inference data flow: one fixed 32-dimension integer vector through a 5-tree ensemble; the audit record is emitted from the decision path, never from the weights.

Why 5 trees after pruning?

Training begins with 300 trees. Post-training pruning identifies the minimal sub-ensemble that preserves accuracy on the validation set within tolerance. The result is 5 trees — small enough to fit in L1 cache on most embedded cores, large enough to provide majority-vote stability (3-of-5 decision boundary). Adding more trees past this point yielded no measurable accuracy gain on held-out data, confirming the ensemble has reached the bias-variance optimum for this feature space.

Why a fixed 32-dimensional feature vector?

Raw syscall sequences are variable-length strings — unusable directly for fixed-size classifiers. The engineering decision is to project any sequence, regardless of length, onto a fixed 32-dimensional space of normalized frequency ratios and statistical aggregates. This projection is the core of what makes the model length-invariant (explained in section 05). The 32-dimensional size is the result of iterative feature selection: it is the smallest dimensionality that retains the inter-class separability needed for the accuracy targets.

Why the training pipeline is a core asset.

An under-appreciated consequence of the tree-ensemble architecture is how cheap it is to train. End-to-end training on the full 5,550-trace dataset takes approximately **13 minutes** on a commodity laptop CPU (4th-generation Intel i7, 8 GB RAM, no GPU) — no accelerator, no cloud quota. This

makes the *trainer*, not merely the model, the differentiating asset: thousands of model variants can be trained, validated, and cross-checked in a single day on hardware that any engineer already owns. Per-deployment tailoring — per device class, per client, per threat landscape — becomes a lunch-break operation rather than a GPU-farm job, the inverse of the deep-learning status quo.

03 Training Dataset

The model was trained on **5,550** real-world syscall traces — **2,797 malware** / **2,753 benign** (50.4% / 49.6%), near-balanced rather than perfectly matched. No synthetic data was used. Traces span 2016–2026, covering a decade of Linux threat evolution with no temporal overfitting observed at evaluation time. The corpus and its full statistical report are published at `OriginalTrianed_dataset`.

Trained once, tested three times — with no retraining.

The ensemble was trained a single time, under a fixed protocol: fixed split, fixed hyperparameters, and no per-corpus tuning. The three evaluations reported in section 06 — in-sample internal accuracy, external Linux static-ELF, and external IoT — all run the *same frozen model*: same weights, same thresholds, same feature definitions, zero retraining, fine-tuning, or dataset-specific adjustment. Generalization is therefore measured under the strict conditions a deployed first-line detector actually faces, not tuned into existence.

Chopped fixed-window sequences — the critical design choice.

Each training sample is not a complete execution trace. It is a fixed-window sub-sequence chopped from a real trace. This is deliberate: by training on bounded windows rather than full runs, the model is forced to learn frequency fingerprints that are statistically stable in any window of sufficient size. The consequence is that the trained model generalizes to traces far longer than anything it saw during training — including traces 51$\times$ the training ceiling — without any retraining.

Split	Samples	Class balance	Length (median / max)
Training set	5,550	2,797 mal / 2,753 benign	863 / 2,275 syscalls

Trace-length distribution by class (raw, from `dataset_stats.json`):

Class	n	Median	Mean	SD	p25	p75	p95	p99	Max
Internal — Malware	2,797	900	973.3	456.7	821	1,030	1,950	2,143	2,200
Internal — Benign	2,753	581	664.4	468.2	301	900	1,665	2,200	2,275
LinuxStatic — Malware	1,477	1,161	1,218.2	940.6	348	1,785	2,852	3,476	12,870
IoT — Malware	1,267	15,863	15,668.7	8,767.6	12,262	19,349	28,289	31,825	117,088
IoT — Benign	238	2,234	6,462.7	11,273.1	1,391	6,910	20,771	57,292	64,389

Read it straight: the internal benign class skews markedly shorter (median 581) than internal malware (median 900) — which is exactly why the short-sequence gate in section 05 is mandatory. The IoT corpus is dominated by long traces (median 15,075, max 117,088); its tiny benign tail (n=238, median 2,234) is the origin of the elevated FPR discussed in section 07.

04 Quantization Trade-off: INT8 vs Float32

The Python training pipeline operates in float32. The deployed C kernel uses INT8 fixed-point throughout — no floating-point unit required at inference time. The quantization step introduces a controlled accuracy loss in exchange for the binary size and latency profile that makes embedded deployment feasible.

What quantization costs.

Across all three evaluation datasets, the INT8 C kernel loses at most 0.5 percentage points in

accuracy compared to the float32 Python model. This is the entire cost of quantization — measured empirically on held-out data, not estimated.

Metric	Float32 (Python)	INT8 (C kernel)	Delta
Accuracy — internal set (in-sample)	98.97%	98.88%	-0.09 pp
Accuracy — LinuxStatic (unseen)	92.42%	92.01%	-0.41 pp
Accuracy — IoT (unseen)	96.48%	96.28%	-0.20 pp
Memory & speed			
Working set per classification	n/a (process-resident)	≈0.03 MB	—
Measured batch-harness peak RAM	966.6–1,001.7 MB	0.125–1.375 MB	—
Median latency (interface-only)	16–388 ms	34–62 ns	—
Binary / environment footprint	≈1 GB env	27 KB	—
FPU required	Yes	No	—

What quantization buys:

- Binary size: 27 KB compiled vs ≈1 GB Python environment.
- Working set: ≈0.03 MB per classification vs ≈1 GB Python environment — an ≈32,000× reduction.
- Latency: 34–62 ns median vs 16–388 ms Python — up to 15.8 million× faster.
- FPU dependency: eliminated entirely on the embedded-Linux deployment target.
- Deployment model: single dependency-free C translation unit, any C compiler, zero runtime.

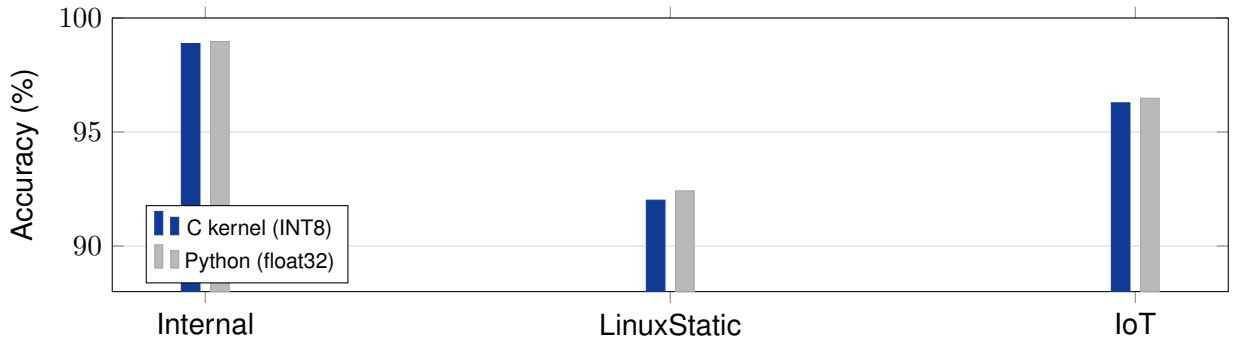

Figure 2: Quantization delta on the three corpora. INT8 C kernel tracks float32 Python within ≤0.5 pp everywhere; the cost of embedded deployability.

Note on the 0.03 MB working set and the earlier 1.4 MB figure: the per-decision working set (≈0.03 MB) is the *steady-state* memory of a single classification — one static model header (27 KB binary) plus one 32-element integer feature vector, no heap allocations on the inference path. The 1.4 MB figure reported earlier was the peak resident set while batch-classifying the *entire* 1,505-trace IoT corpus inside a single process — a lab artifact in which the whole dataset is held in memory at once. The per-decision figure is the deployment-relevant number; the measured batch harness peaks (0.125 MB internal / 0.5 MB LinuxStatic / 1.375 MB IoT) are reproduced in the benchmark reports.

05 Out-of-Distribution Generalisation: Why Length-Invariance Works

The model was trained once, on traces up to 2,275 syscalls. On the external IoT corpus — whose longest traces reach 117,088 syscalls, a 51× extrapolation beyond the training ceiling — the same frozen model achieves 97.87% recall and 96.28% accuracy with no retraining or fine-tuning. This result requires explanation.

The mathematical intuition.

The 32-dimensional feature vector is built entirely from normalized frequency ratios and statistical aggregates, not from raw counts or positional features. A normalized ratio — such as the proportion of file-I/O syscalls out of total syscalls, or the Shannon entropy of the call distribution — converges to

a stable value as the sequence grows and remains statistically consistent across any window once a minimum sample size is reached.

Concretely: if a malware process makes file-I/O calls 54% of the time, that ratio is approximately 0.54 whether measured over 500 syscalls or 50,000. The feature vector captures what kind of process this is, not how long it ran. The decision boundary learned during training on short windows transfers intact to long traces because the underlying statistical signature is the same.

Why this also explains the short-trace failure mode.

The same property that enables long-trace generalization also defines the failure regime. Frequency ratios are estimates from a sample. Below $\sim$500 syscalls, the sample is too small for the ratio to stabilize — high-variance estimates push the feature vector away from its true population value and across the decision boundary. This is a statistical convergence issue, not a model capacity issue, and it is why the minimum-length gate is mandatory rather than optional.

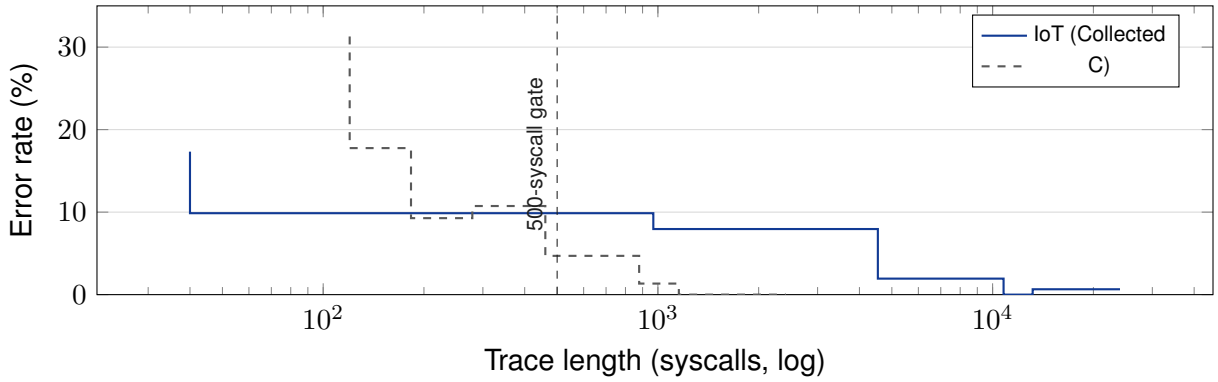

Figure 3: Error rate vs trace length (C kernel). Both external corpora confirm the same curve: errors concentrate on very short traces and vanish once length stabilizes — the quantitative basis for the 500-syscall gate.

Trace length range	Feature ratio stability	Error rate	Action
$<$ 500 syscalls	Unstable — high variance	16%–30%	Buffer; do not classify
500–1,200 syscalls	Converging	5%–10%	Classify with caution
1,200–2,275 syscalls	Stable (training regime)	$<$ 2%	Optimal window
$>$ 2,275 syscalls	Stable (OOD, confirmed)	$<$ 2%	Generalises well

Error counts by trace-length bin (measured, C kernel — what drives the curve above):

IoT (Collected) — C kernel			
Length range	**n**	**Errors**	**Error rate**
40–969	150	26	17.33%
969–4,541	152	15	9.87%
4,541–10,783	151	12	7.95%
10,783–13,172	155	3	1.94%
13,172–24,019	461	0	0.00%
24,019–117,088	154	1	0.65%
LinuxStatic — C kernel			
Length range	**n**	**Errors**	**Error rate**
120–183	147	46	31.29%
183–279	152	27	17.76%
279–461	151	14	9.27%
461–879	149	16	10.74%
879–1,155	149	7	4.70%
1,155–2,414	596	8	1.34%
2,414–12,870	149	0	0.00%

06 Benchmark Results

One model, three evaluations. Planck-99 was trained exactly once; the three evaluations below are three separate runs of that single frozen model, without any retraining or per-dataset tuning — deliberately the strict conditions under which a deployed first-line detector actually operates. The internal set measures in-sample (resubstitution) accuracy — the model is re-scored on the traces it was trained on, so it is a *calibration/regression* number, not a generalization number; generalization is measured on the two fully external corpora (zero overlap with training data). All results are from the C kernel (INT8) unless noted. The full per-corpus reports (results, statistics, and error analysis) are the primary references of this brief and are published in the three dataset folders of the public benchmark repository (links below).

Dataset	Type	Accuracy	Precision	Recall	F1	FPR
Training set (internal)	In-sample (resub.)	98.88%	98.58%	99.21%	98.90%	1.45%
LinuxStatic ELF	External — unseen	92.01%*	100.00%	92.01%	95.84%	**
IoT Syscalls (Collected)	External — unseen	96.28%	97.71%	97.87%	97.79%	12.18%

*Detection rate on a single-class corpus (see below). **Not measurable — the corpus contains no benign traces, so FPR = 0/0 is undefined; accuracy and precision are trivially tied to the malware class (precision 100% follows from zero false positives with no benign predictions possible).

Collected dataset profile.

Collected is an independently collected IoT syscall corpus (`IoTsyscalls`), captured under device and workload conditions distinct from training, with zero trace overlap on the training data — it serves here as a stress-test for out-of-distribution generalization, not as a balanced benchmark. The evaluated subset contains 1,505 samples with a heavily skewed 1:5.3 benign-to-malware ratio — only 238 benign traces against 1,267 malware traces. Benign traces have a median of 2,234 syscalls; malware extends to 117,088 (51× the training ceiling of 2,275).

LinuxStatic ELF.

A disjoint external corpus of syscall traces from Linux static-ELF malware (`linuxStaticSyscalls_MALWARES`), collected independently of training — the stringent case in which a false positive costs an alert and a miss costs the air-gap. Because every one of the 1,477 traces is malware, the *only* honest headline is the detection rate (recall): **92.01%**. Precision and FPR are structurally unavailable on a single-class corpus and are reported as such rather than implied.

Full latency percentiles (C kernel — 9 independent runs; Python interface-only).

Dataset	C kernel (ns)				Python (ms)			Speedup
	p50	p90	p99	max	p50	p95	p99	
Training set (internal)	62	68	73	73	16.41	21.38	27.12	≈274,000×
LinuxStatic ELF	35	39	44	44	16.09	26.76	50.27	≈510,000×
IoT Syscalls (Collected)	34	39	41	41	388.08	1,334.66	2,508.64	≈15,800,000×

Latency figures are for the classifier interface only (fixed 32-feature input, median of 9 runs, $\sigma \leq 14$ ns); they exclude trace capture, feature extraction, and audit serialization. Latency is trace-length invariant by design — the kernel receives the same fixed feature vector regardless of original trace length. Python figures are 'pipe.predict([seq])' on a pre-loaded model.

Confidence intervals (exact Clopper–Pearson, 95%).

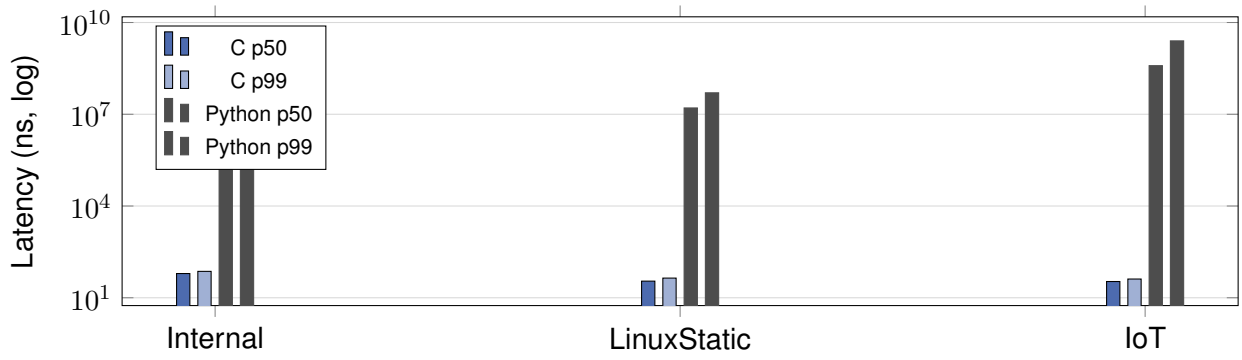

Figure 4: Latency distribution (log scale): C kernel 34–62 ns vs Python 16–388 ms across all corpora.

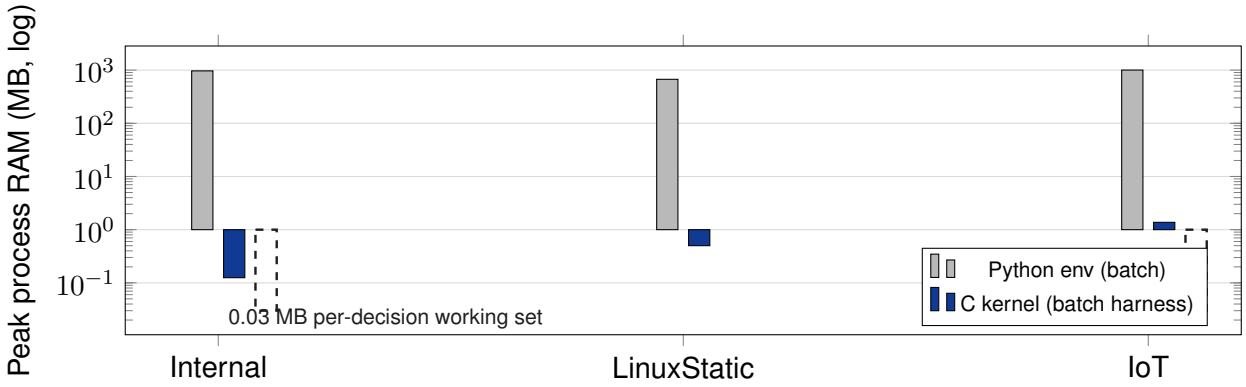

Figure 5: Peak process RAM (log scale): Python 0.7–1.0 GB vs C harness 0.125–1.375 MB batch, ≈0.03 MB per single decision.

Quantity	Point estimate	95% CI	Source
IoT FPR (29/238)	12.18%	8.31%–17.03%	IoTsyscalls
IoT accuracy (1,449/1,505)	96.28%	95.20%–97.18%	IoTsyscalls
Internal accuracy (5,488/5,550)	98.88%	98.57%–99.14%	OriginalTrianed_dataset
Internal recall (2,775/2,797)	99.21%	98.81%–99.51%	OriginalTrianed_dataset
LinuxStatic detection rate (1,359/1,477)	92.01%	90.51%–93.34%	linuxStaticSyscalls_MALWARES

Numbers you can trust. Every figure above is reproduced from the published `results.json`, `dataset_stats.json`, and `error_analysis.json` in the three benchmark folders — no number in this brief is a rehearsal or a rounded guess from memory. Accuracy is exact confusion-matrix arithmetic (e.g., 5,488/5,550 = 98.88%); latency is the median of 9 independent runs measured with the C-internal clock; the CIs are exact Clopper–Pearson bounds, not normal approximations.

07 Known Limitations

L1 — Full sequence required per classification pass.

Planck-99 v1 classifies a complete buffered window, not an incrementally updated stream. A process must accumulate at least 500 syscalls before any verdict is possible, and the recommended window is 1,200+ syscalls for reliable operation. The deployment model is: collect → gate → classify → slide window by 50% → repeat. This means early-lifecycle processes cannot be classified until sufficient evidence accumulates. Future versions will address this with an online feature-update architecture.

L2 — 12.18% false positive rate on IoT benign traces.

On Collected the model raises 29 false alarms from 238 benign samples (12.18%; exact 95% CI 8.31–17.03%). The root cause is a distribution mismatch, not a model flaw:

- The 238 benign traces have a median of only 2,234 syscalls — within the converging (not converged) region of the frequency-ratio window. The fingerprint has not fully stabilized, pushing some benign samples toward the malware side of the decision boundary.
- The benign traces are dominated by generic system calls (read, mmap, fstat) whose frequency profiles overlap with certain malware behavioral signatures at short lengths.
- The 1:5.3 class imbalance in Collected means benign is severely underrepresented; on the balanced internal dataset FPR is 1.45%.

Accepted trade-off rationale: in the target deployment context, a missed malware detection carries significantly higher operational cost than a false alarm. A false positive triggers a review event; an undetected intrusion on an air-gapped embedded device may be unrecoverable. At this model size, latency, and accuracy profile, the v1 FPR is an accepted trade-off for a first-line detector.

L3 — No incremental / streaming classification in v1.

The current architecture requires a complete window for each inference pass. Streaming updates to the feature vector as syscalls arrive would allow earlier detection and eliminate the buffering latency. This is a planned v2 capability.

L4 — Packed / anti-debug malware blind spot.

On LinuxStatic the missed traces have a markedly shorter median length (205 syscalls vs 1,216 correct) and concentrate in the hard-to-classify short regime. Among the top syscalls over-represented in false negatives are `mprotect`, `ptrace`, and `mmap` — the signature blend of packed, self-modifying, or anti-debugging binaries whose early execution footprint differs from the behavioral pattern the model learned. These are a first-line detection gap, not an evasion of the evaluator: the feature model simply has not seen enough short packed-malware examples (see figure in section 05).

L5 — Metrics scope: pre-gate vs post-gate, and end-to-end latency.

Reported accuracy includes traces shorter than the 500-syscall deployment gate (the gate is applied in production, not retroactively in the benchmarks). Applying the gate post-hoc would raise measured accuracy but mis-state what a real device sees; accordingly the brief reports the *pre-gate* numbers and documents the error-by-length curve so deployers can quantify the effect of their own buffering. Latency is the classifier interface only: capture adds a syscall-logger cost that depends on the hook (≈0.1–1 μs/event for eBPF, 0.5–2 μs/event for LTTng, tens of μs under strace). On-device end-to-end latency is dominated by capture, not by the 34–62 ns decision.

08 Deployment Model

The intended deployment is a lightweight controller loop running alongside the kernel syscall logger on any embedded Linux SoC. Total static footprint is the 27 KB C kernel binary plus a small ring buffer sized to the buffered window (600 events at a few bytes each as compact syscall IDs) — fits in the SRAM of any modern embedded Linux device without a dedicated AI accelerator.

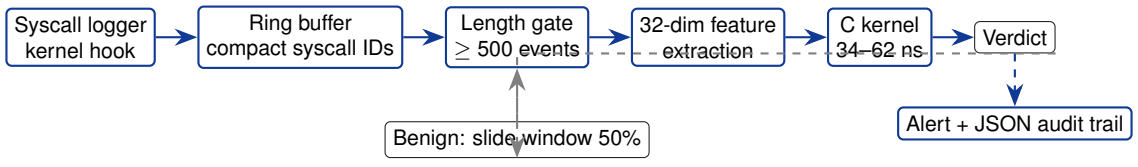

Figure 6: Deployment loop: fully on-device; no network, no FPU, no runtime. Only alert records leave the device.

Step	Component	Detail
1	Syscall logger	Kernel hook appends each syscall to a ring buffer
2	Length gate	Buffer $<$ 500 syscalls $\rightarrow$ keep buffering, no classification
3	Feature extraction	32-dimensional normalized vector computed from buffer
4	C kernel inference	34–62 ns; $\approx$0.03 MB working set; integer-only, zero FPU, zero network
5a	On Malware verdict	Raise alert + write JSON audit trail
5b	On Benign verdict	Slide window by 600 syscalls (50% overlap); go to step 2

Operational and compliance posture.

- **Audit trail as evidence, not logic.** Each record carries input-feature hash, decision-path node IDs, threshold references/hashes, verdict, and pinned engine version — so the record is tamper-evident and replayable without disclosing model internals. Records are append-only and hash-chained on rotation.
- **Retention and opt-out.** Logs are retained per applicable retention law (e.g., GDPR Art. 5(1)(e) purposes-limited storage) and are subject to a genuine *feature-level* opt-out (disabling classification of a given process), not a generic logging opt-out. Under NIS2 covering an essential system, opt-out is disabled by design.
- **Supply-chain hygiene.** Engine versions are pinned, a software bill of materials (SBOM) accompanies each release, and clock-skew is removed from the decision path in any case (no timing dependence); NTP sync is used only for audit time-stamping.
- **Vulnerability disclosure.** Security findings follow a coordinated disclosure (CVD) route with a public security contact and no back-door requirements.

09 Competitive Position

Capability	Traditional AV	Cloud detection	Planck-99 v1
Working set (per classification)	50–500 MB	Server-side	**0.03 MB**
Inference latency	10–100 ms	200–2,000 ms	**34 ns**
Network required	No (offline sig)	Mandatory	**Never**
FPU / GPU required	No / No	Yes / Yes	**No / No**
Validated platform	Desktop / server	Server	**Embedded Linux (MCU: future work)**
Evaluation protocol	Vendor-reported	Vendor-reported	**Trained once, 3 unseen corpora**
Model training cost	Vendor signature labs	GPU cluster	**$\approx$13 min, CPU-only**
Per-deployment model	Weekly signature push	Vendor-managed	**Train on demand**
Air-gap safe	No	No	**Yes**
Per-decision audit	No	No	**JSON proof trail**
Accuracy (internal, in-sample)	Vendor-reported	N/A	**98.88%**
Accuracy on unseen IoT	95–98% (est.)	N/A	**96.28%**
False positive rate	Vendor-dependent	N/A	**1.45% (balanced) / 12.18% (skewed)**

Two rows in the table above capture the strategic difference that follows directly from section 02. Because end-to-end training runs in roughly thirteen minutes on a single CPU, the *trainer* out-sells signature feeds: Planck-99 models are regenerated per deployment, per device class, or per client — and thousands of candidate versions can be trained and stress-tested in a day, on hardware that already exists in the customer's engineering team. This is the inverse of the deep-learning cost model, where training is the expensive step and inference is cheap.

10 Roadmap

Priority	Improvement	Target outcome
High	Online / streaming feature updates	Verdicts without fixed-window buffering
High	Expanded benign training data (IoT)	FPR $<$ 3% on short benign traces
High	On-demand per-deployment training service	Tailored models generated in minutes on commodity CPU
Medium	ARM / RISC-V embedded benchmarks	Validated latency on MCU-class hardware
Medium	Multi-class detection (v2)	Malware family identification
Low	Relaxed minimum sequence gate	Reliable inference below 500 syscalls
Low	Packed-malware training samples	Close the short-trace `mprotect`/`ptrace`/`mmap` gap

11 References — Public Benchmarks

The primary references of this brief are the three dataset folders of the public benchmark repository. Each folder contains its own `results.json`, `dataset_stats.json`, `error_analysis.json`, `summary.txt`, and full result figures. All numbers reported in this brief are reproduced from these reports:

- `github.com/Division-36/Planck-99_PublicBenchmarks/tree/main/OriginalTrianed_dataset` — training corpus (5,550 traces, 2016–2026) and internal evaluation report.
- `github.com/Division-36/Planck-99_PublicBenchmarks/tree/main/IoTsyscalls` — external IoT syscall evaluation report (unseen, OOD).
- `github.com/Division-36/Planck-99_PublicBenchmarks/tree/main/linuxStaticSyscalls_MALWARES` — external Linux static-ELF malware evaluation report (unseen, single-class).

Repository root: github.com/Division-36/Planck-99_PublicBenchmarks.